

HANDOVER · INTERNAL

데이터베이스/파이프라인 설계서

문서번호 DUBBY-DPD-001

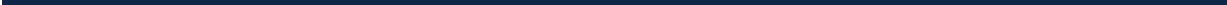
제품 버전 3.0.12

파이프라인 버전 3.0

작성일 2026-09-05

상태 **인수인계용 / 내부**

이 문서는 Dubby의 스키마, 잡 튜, 크레딧 원장, R2 키, 미디어 파이프라인 단계를 정의한다. 런타임 체계는 SAD-001, 제품 저장소 코드 마이그레이션 기준 (추측 데이터 없음).



1. 문서 개요

1.1 목적

이 문서는 Dubby 3.0.12의 데이터 저장과 워커 파이프라인을 인수인계 기준으로 고정한다. 어떤 테이블이 실제로 존재하는지, 잡이 어떻게 claim-heartbeat-timeout 되는지, transcribe/dub/lipsync가 어떤 단계와 진행 코드를 쓰는지, R2 객체가 DB 어느 컬럼을 가리키는지를 코드와 SQL 마이그레이션에 근거해 기술한다.

1.2 범위

- 포함: Repository 백엔드, supabase/migrations/ 스키마, 상태 문자열, jobs 큐, 크레딧 원장 행·불변식, R2 키 레이아웃, transcribe/dub/lipsync 단계, 스크래치·재시도·취소, 3.0.12 데이터 함의, 마이그레이션 적용 방법.
- 제외: Caddy/CORS/터널, Stripe Checkout 화면, 크레딧 요금 UX, STT·번역·TTS 프롬프트 전문. 이들은 SAD-001, FRS-001, 후속 「AI 에이전트 및 프롬프트 명세서」에서 다룬다.
- 사실의 기준: 저장소 현재 코드(제품 3.0.12, PIPELINE_VERSION = "3.0"). 마이그레이션에 없는 테이블·컬럼은 만들지 않고, 코드에만 있는 필드는 사이드카로 명시한다.

1.3 관련 문서

문서	역할
시스템 아키텍처 정의서 (DUBBY-SAD-001)	런타임·배포·네트워크·저장 경계
비즈니스 로직 및 기능 명세서 (DUBBY-FRS-001)	제품 기능, 크레딧 UX, 추출/더빙 화면
본 문서 (DUBBY-DPD-001)	스키마, 잡, 미디어 단계, R2 키
AI 에이전트 및 프롬프트 명세서 (후속)	STT·번역·TTS 프롬프트와 모델
infra/DEPLOY.md	supabase db push 부트스트랩
VERSION.md	3.0 / 3.0.12 파이프라인 메모

1.4 용어

용어	의미
Repository	api/app/db/ 추상화. API·워커는 이 인터페이스만 사용한다.
Row	컬럼명→값 dict. 라우터가 Pydantic으로 검증한다.
service role	API·워커가 쓰는 권한. RLS를 우회하므로 owner 스코프는 쿼리에 넣는다.
사이드카	DB 컬럼이 없거나 미적용일 때 R2 meta/ JSON으로 보완하는 값.
assign-by-idx	번역 결과를 세그먼트 idx로만 매핑. 위치 재배정은 서버 파이프라인에서 금지.

용어	의미
heartbeat_at	실행 중 잡의 생존 시각. 진행 갱신마다 now()로 덮어쓴다.

2. 데이터 접근 경로

API와 워커는 저장소에 직접 SQL을 흘뿌리지 않고 `create_repository(settings)`가 고른 Repository만 사용한다. 백엔드는 `Settings.db_backend`로 갈린다. 기본값은 `postgres`이다.

```
[FastAPI / worker.runner]
|
v
create_repository(settings)
|
+-- db_backend == "postgres"      --> PostgresRepository (asyncpg)
|                                DATABASE_URL, session-mode pooler
|
+-- db_backend == "supabase_rest" --> SupabaseRestRepository (httpx)
|                                SUPABASE_URL + SERVICE_ROLE_KEY
|                                PostgREST + SECURITY DEFINER RPC
```

2.1 postgres (권장)

`api/app/db/postgres.py`. Lightsail에서 Supabase connection pooler URI(session mode)로 붙는다. `asyncpg.create_pool` 파라미터는 `config.py`에서 온다.

설정	기본값	역할
<code>db_backend</code>	<code>postgres</code>	백엔드 선택
<code>database_url</code>	(빈 문자열)	production + postgres면 필수
<code>db_pool_min_size</code>	1	풀 하한
<code>db_pool_max_size</code>	5	풀 상한
<code>command_timeout</code>	30초 (코드 상수)	풀 생성 시 고정

기동 시 `information_schema.columns`로 `projects` 실존 컬럼을 읽어 SELECT 목록을 줄인다. 아직 적용되지 않은 마이그레이션 컬럼이 있어도 `list_projects`가 500을 내지 않는다. 빠진 컬럼은 로그 경고만 남긴다.

2.2 supabase_rest

`api/app/db/supabase_rest.py`. 직접 DB 포트가 없을 때 PostgREST + service-role 키로 동일 인터페이스를 구현한다. 원자 작업은 SQL RPC에 맡긴다.

작업	RPC / 경로
잔액	<code>credit_balance(p_user_id)</code>
enqueue + 차감	<code>enqueue_job_with_credit</code>
claim	<code>claim_next_job()</code> — FOR UPDATE SKIP LOCKED
세그먼트 일괄 교체	<code>replace_segments(p_project_id, jsonb)</code>

작업	RPC / 경로
세그먼트 편집	update_segment_texts
종료+환불	finish_job_with_refund
stale fail	fail_stale_jobs(p_timeout_seconds)
Stripe / RevenueCat	process_stripe_event / process_revenuecat_event

클라이언트 역할(anon, authenticated, PUBLIC)에는 위 RPC EXECUTE가 revoke되어 있다. dub_voice_ids 컬럼이 아직 없으면 REST 백엔드는 R2 사이드카 dub_voice_ids.json으로 보완한다.

2.3 소유권 규칙

base.py: 사용자 대면 메서드는 모두 owner_id를 받고 그 범위로 쿼리해야 한다. API는 privileged credential로 붙으므로 RLS만으로 소유권을 맡기지 않는다. 워커 메서드 (claim_next_job, get_project_for_worker, replace_segments 등)는 owner 스코프가 없다.

3. 실제 스키마

단일 진실 원천은 supabase/migrations/이다. 초기 파일 20260717000000_init_dubby_core.sql 이후 ALTER가 누적된다. 아래는 **최종 적용 후** 형상이다. 마이그레이션에 없는 필드는 3.10절에 따로 적는다.

3.1 마이그레이션 이력

파일	요지
20260717000000_init_dubby_core.sql	profiles, projects, segments, jobs, credit_ledger, waitlist, RLS, handle_new_user, 초기 RPC
20260717010000_worker_pipeline_rpc.sql	replace_segments
20260717020000_stripe_credits.sql	jobs.charged_minutes, ledger 불변-idempotency, Stripe 3테이블, enqueue/finish RPC
20260717030000_phase2_quality.sql	tone/diarize/warnings/lipsync, jobs.kind lipsync, lipsync_job
20260717040000_revenuecat_mobile_billing.sql	RevenueCat 이벤트·구독 투영
20260717050000_segment_source_text_edit.sql	update_segment_texts에 source_text
20260718143000_auth_admin.sql	profiles 국가/provider, access_logs, ledger admin 컬럼
20260719165000_bootstrap_first_admin.sql	지정 이메일 1건에 app_metadata.role=admin
20260723120000_admin_user_active.sql	profiles.is_active / deactivated_at
20260723140000_languages_and_signup_credits.sql	언어 확장, signup 30분
20260730000000_credit_ledger_allow_fk_null.sql	프로젝트 삭제 시 FK null만 허용
20260806000000_user_voices.sql	My Voice Box
20260807000000_project_dub_voice_ids.sql	projects.dub_voice_ids jsonb
20260808000000_pipeline_v2_voice_mode.sql	voice_mode, pipeline_version (기본 2.0)
20260818000000_segment_speak_speed.sql	segments.speak_speed, end_ms 편집
20260823000000_add_th_my_languages.sql	th, my CHECK
20260824000000_tone_style_emotions.sql	7감정 + legacy tone CHECK, 기본 calm

3.2 public.profiles

auth.users와 1:1. handle_new_user 트리거가 INSERT한다.

컬럼	형	제약
id	uuid PK	auth.users(id) ON DELETE CASCADE
email	text	
display_name	text	메타 full_name/name/user_name 또는 로컬파트
country	text	NULL 또는 <code>^[A-Z]{2,8}\$</code>
auth_provider	text	app_metadata.provider, 기본 email
last_login_at	timestampz	
is_active	boolean NOT NULL	기본 true. false면 비관리자 API 거부
deactivated_at	timestampz	
created_at / updated_at	timestampz	updated_at 트리거

RLS: 본인 SELECT/UPDATE. 인덱스: `profiles_is_active_idx` (`is_active=false` partial).

3.3 public.projects

컬럼	형	제약 / 비고
id	uuid PK	gen_random_uuid()
owner_id	uuid NOT NULL	profiles(id) ON DELETE CASCADE
title	text NOT NULL	길이 1-200
status	text NOT NULL	기본 created. CHECK: created, uploading, uploaded, processing, ready_for_edit, dubbing, completed, failed
source_lang / target_lang	text NOT NULL	기본 ko / en. CHECK: ko,vi,en,zh,ja,es,fr,pt,de,ru,ar,ur,id,ms,tr,ta,th,my
subtitle_mode	text NOT NULL	none source target (기본 target; API 생성 기본은 none)
tone_style	text NOT NULL	기본 calm. sad,angry,whisper,excited,energetic,calm,cheerful + legacy neutral,warm,serious
diarization_enabled	boolean NOT NULL	기본 false
dub_voice_ids	jsonb NOT NULL	기본 []. 화자 슬롯용 ElevenLabs voice ID 배열(최대 8, 코드)
voice_mode	text NOT NULL	voice_box auto_clone. 기본 voice_box

컬럼	형	제약 / 비고
pipeline_version	text NOT NULL	SQL 기본 2.0. API 응답은 요청/사이드카로 3.0을 덮어씀
duration_seconds	double precision	NULL 또는 >= 0. transcribe probe 후 기록
source_key	text	R2 원본 키
output_key	text	더빙 mp4 키
lipsync_output_key	text	립싱크 mp4 키
quality_warnings	jsonb NOT NULL	기본 []. 배열만 허용
error	text	소프트 삭제 센티널 __deleted__도 여기
created_at / updated_at	timestampz	

인덱스: projects_owner_created_idx (owner_id, created_at DESC). RLS: 본인 CRUD. 목록 쿼리는 coalesce(error, '') <> '__deleted__'로 숨긴다.

pipeline_version 기록 불일치

SQL 기본값과 PostgresRepository.create_project INSERT는 컬럼이 있으면 "2.0"을 넣는다. 라우터는 본문을 3.0으로 응답하고 R2 meta에 저장한다. DB 컬럼만 보면 3.0 프로젝트도 2.0으로 남을 수 있다.

3.4 public.segments

컬럼	형	제약 / 비고
id	uuid PK	
project_id	uuid NOT NULL	projects(id) ON DELETE CASCADE
idx	integer NOT NULL	>= 0. UNIQUE(project_id, idx)
start_ms	integer NOT NULL	>= 0
end_ms	integer NOT NULL	end_ms > start_ms. 더빙 후 슬롯 연장·speak_speed로 바뀔 수 있음
source_text / target_text	text NOT NULL	기본 "
speaker_id	text	예: speaker_0, speaker_1

컬럼	형	제약 / 비고
speaker_overlap	boolean NOT NULL	기본 false
speak_speed	double precision	편집기/파이프라인 TTS 배율. 1.0=자연
created_at / updated_at	timestampz	

인덱스: segments_project_idx (project_id, idx). RLS: 프로젝트 소유자 SELECT/UPDATE. INSERT/DELETE는 파이프라인(service role)만. replace_segments는 해당 프로젝트 행을 전부 지운 뒤 다시 넣는다. 워커 INSERT 컬럼은 idx, start_ms, end_ms, source_text, target_text, speaker_id, speaker_overlap이다. speak_speed는 이후 편집·더빙 persist로 갱신된다.

3.5 public.jobs

컬럼	형	제약 / 비고
id	uuid PK	
project_id	uuid NOT NULL	projects(id) ON DELETE CASCADE
kind	text NOT NULL	transcribe dub lipsync
status	text NOT NULL	queued running completed failed cancelled. 기본 queued
progress	double precision NOT NULL	0-1. 기본 0
message	text	진행 코드. UI는 job-labels.ts로 번역
error	text	실패 시 code: message 또는 worker timeout
charged_minutes	numeric(10,2) NOT NULL	>= 0. 기본 0. dub/lipsync 선차감액
started_at / finished_at / heartbeat_at	timestampz	claim-진행-종료 시 기록. API JobOut에는 없음
created_at / updated_at	timestampz	

인덱스:

- jobs_project_created_idx (project_id, created_at DESC)
- jobs_queue_idx (status, created_at) WHERE status IN ('queued', 'running')
- jobs_one_active_per_project UNIQUE(project_id) WHERE status IN ('queued', 'running')

RLS: 소유자 SELECT만. 쓰기는 API/워커. API SELECT 화이트리스트는 id, project_id, kind, status, progress, message, error, created_at, updated_at이다.

3.6 public.credit_ledger

추가만 가능한 원장. 잔액 = `sum(delta_minutes)`.

컬럼	형	제약 / 비고
id	uuid PK	
user_id	uuid NOT NULL	profiles(id) ON DELETE CASCADE
delta_minutes	numeric(10,2) NOT NULL	< > 0
reason	text NOT NULL	signup_grant, dub_job, lipsync_job, refund, purchase, admin_adjust
project_id	uuid	projects ON DELETE SET NULL
job_id	uuid	jobs ON DELETE SET NULL
external_reference	text	Stripe/RevenueCat 거래 참조
idempotency_key	text	부분 UNIQUE (NOT NULL일 때)
admin_note / adjusted_by	text / uuid	관리자 조정. adjusted_by → profiles SET NULL
created_at	timestampz	

부분 유니크: job debit(reason='dub_job'), lipsync debit, job refund, idempotency_key.
UPDATE/DELETE 트리거 `credit_ledger_immutable` — 부모 삭제 시 `project_id/job_id`를 NULL로 만드는 것만 허용. RLS: 본인 SELECT. INSERT 정책 없음(서버만).

3.7 public.user_voices

컬럼	형	제약
id	uuid PK	
owner_id	uuid NOT NULL	auth.users(id) ON DELETE CASCADE (profiles가 아님)
nickname	text NOT NULL	trim 1–30. UNIQUE(owner_id, nickname)
elevenlabs_voice_id / shared_voice_id	text NOT NULL	UNIQUE(owner_id, shared_voice_id)
public_owner_id, name, description, gender, accent, category, language, age	text	기본 "
preview_url	text	
created_at	timestampz	

인덱스: user_voices_owner_created_idx. RLS: 본인 CRUD. 중복 시 코드가 DuplicateVoiceError.

3.8 Stripe / RevenueCat / access_logs / waitlist

테이블	PK	요지
stripe_customers	user_id → profiles	stripe_customer_id UNIQUE. 본인 SELECT
stripe_subscriptions	stripe_subscription_id	status, price_id, period_end, cancel_at_period_end. 본인 SELECT
stripe_events	stripe_event_id	event_type, payload jsonb. 클라이언트 정책 없음
revenuecat_events	revenuecat_event_id	웹훅 inbox. 서비스 전용
revenuecat_customers	app_user_id	user_id UNIQUE. 본인 SELECT
revenuecat_subscriptions	(user_id, entitlement_id)	product/status/expires/store. 본인 SELECT
access_logs	id	method, path, status_code, ip inet, user_agent. 클라이언트 정책 없음
waitlist	id	email UNIQUE + 형식 CHECK. anon INSERT. 현재 API 코드는 이 테이블을 읽거나 쓰지 않는다

별도 webhooks 테이블은 없다. 웹훅 원장은 stripe_events와 revenuecat_events이다.

3.9 함수·트리거

이름	역할
set_updated_at	BEFORE UPDATE → updated_at = now()
handle_new_user	auth.users INSERT 후 profile + signup_grant 30분 (on conflict do nothing)
prevent_credit_ledger_mutation	원장 불변. FK nulling만 예외
enqueue_job	초기 RPC. 크레딧 없음. 코드 경로는 enqueue_job_with_credit / 클라이언트 트랜잭션
enqueue_job_with_credit	프로젝트·잔액 FOR UPDATE, active_job_exists, debit
claim_next_job	가장 오래된 queued → running, started_at/heartbeat_at = now()
finish_job_with_refund	failed/cancelled이고 charged_minutes>0이면 refund 행
fail_stale_jobs	heartbeat 초과 running을 finish_job_with_refund(failed, 'worker timeout')

3.10 코드에만 있는 필드 (마이그레이션 없음)

아래는 CREATE TABLE/ALTER TABLE에 없다. SegmentOut-편집기-워커 Row dict와 R2 매니페스트에만 존재한다.

필드	어디에 있는가
source_end_ms	R2 dub_voice_manifest.json. 원본 ASR 끝. end_ms와 달라질 수 있음. DB 컬럼 아님
emotion_tone / emotion_user_set	동일 매니페스트. 세그먼트 감정 오버라이드
clip_speak_speed	매니페스트. 실제 TTS에 쓴 배율
baseline_speak_speed	API/프론트 파생. 보통 1. 저장 컬럼 없음
dubbed_audio_url	메타 클립 키의 프리사인 GET. DB에 URL을 저장하지 않음
gain_db, source_level_db, tts_level_db	매니페스트. 미리보기 음량
audio_key	매니페스트. meta/dub_seg_{idx}.wav 등

list_segments SQL은 id, project_id, idx, start_ms, end_ms, source_text, target_text, speaker_id, speaker_overlap, speak_speed만 읽는다. 나머지는 enrich_segments_with_dub_voice가 붙인다.

4. 핵심 엔터티 관계

```
auth.users (Supabase Auth)
  | 1:1 trigger handle_new_user
  v
profiles -----+
  |               |
  | 1:N           | 1:N
  v               v
projects 1:N segments      credit_ledger
  | 1:N           | (project_id, job_id nullable)
  v               |
jobs <-----+
  |
  +-- kind transcribe | dub | lipsync

profiles 1:1 stripe_customers
profiles 1:N stripe_subscriptions
profiles 1:N access_logs          (user_id SET NULL)
auth.users 1:N user_voices        (owner_id → auth.users)

revenuecat_customers.app_user_id → profiles.user_id
revenuecat_subscriptions (user_id, entitlement_id)

stripe_events / revenuecat_events : 사용자 FK 없음 (이벤트 inbox)
waitlist : 고립 테이블 (현재 API 미사용)
```

사용자는 Supabase Auth UUID이다. 공개 프로파일은 profiles. 프로젝트 미디어 바이트는 Postgres에 없고 R2 키만 둔다. 세그먼트 미리보기 오디오도 R2 메타이다.

5. 프로젝트·세그먼트·잡 상태 머신

문자열은 코드·CHECK와 동일하다. 다른 값을 만들지 않는다.

5.1 프로젝트 status

```
created --POST /v1/projects (DB default)
|
| multipart create 또는 URL ingest 시작
v
uploading -- source_key 기록
|
| multipart complete / ingest 성공
v
uploaded
|
| transcribe claim
v
processing ---- fail ----> failed
|
| replace_segments 성공
v
ready_for_edit -- 취소 시 여기로 복귀 (dub)
|
| dub claim
v
dubbing ---- fail ----> failed
|
| output_key 업로드
v
completed (lipsync는 상태를 바꾸지 않고 lipsync_output_key만 기록)
```

소프트 삭제: status=failed, error='__deleted__', 키 NULL
목록에서 제외

전이

누가

created → uploading	uploads.create_multipart, URL ingest
uploading → uploaded	uploads.complete, ingest 성공
* → processing	run_transcribe 시작. output/lipsync 키 클리어
processing → ready_for_edit	transcribe 성공
ready_for_edit → dubbing	run_dub 시작. lipsync_output_key 클리어
dubbing → completed	run_dub 성공 + output_key
processing/dubbing → failed	PipelineError 또는 INTERNAL
transcribe 취소	processing이 아니면 이전 status, 아니면 uploaded
dub 취소	ready_for_edit

5.2 잡 status


```
queued --claim_next_job--> running --finish_job--> completed
|                                     failed      (error 설정, 환불 가능)
+-----> cancelled (사용자 또는 이미 비-running)
```

사용자 취소: POST /v1/jobs/{id}/cancel → finish_job(..., status="cancelled", error="cancelled by user"). 워커는 다음 checkpoint에서 JobCancelled를 올리고 status를 덮어쓰지 않는다.

5.3 세그먼트

세그먼트 행에 status 컬럼은 없다. 프로젝트 status와 잡이 수명을 나타낸다. transcribe는 전량 교체. 편집은 target_text, source_text, end_ms, speak_speed. source_end_ms-emotion은 R2.

6. Job 모델

6.1 kind

API JobKind와 DB CHECK: transcribe | dub | lipsync. 핸들러 맵 PIPELINE_HANDLERS와 같다. transcribe는 크레딧 0. dub/lipsync는 $\text{ceil}(\text{duration_seconds}/60 * \text{multiplier})$ 를 선차감한다. lipsync는 lipsync_provider != disabled이고 output_key가 있어야 한다.

6.2 claim / poll

python -m app.worker.runner. 루프는 세마포어를 잡고 claim_next_job을 호출한다. 없으면 worker_poll_interval_seconds(기본 3.0) 대기. claim SQL:

```
UPDATE jobs SET status='running', started_at=now(), heartbeat_at=now()
WHERE id = (
  SELECT id FROM jobs WHERE status='queued'
  ORDER BY created_at
  FOR UPDATE SKIP LOCKED
  LIMIT 1
)
```

worker_concurrency 기본 1, 상한 8. 주석: Demucs 메모리 때문에 소형 Lightsail에서는 1.

6.3 heartbeat와 stale reaper

항목	값 (코드)
진행 갱신	UPDATE ... heartbeat_at=now() WHERE status='running'
장시간 서브프로세스	ctx.heartbeat() — 마지막 progress/message 재기록
pipeline_heartbeat_seconds	20.0 (엔진이 긴 ffmpeg/Demucs 중 호출)
worker_job_timeout_seconds	3600
reaper 주기	60초 (_STALE_REAP_INTERVAL_SECONDS)
stale 조건	status=running AND heartbeat_at < now() - 3600s
stale 결과	failed, error=worker timeout, charged면 refund
프로세스 하트비트 파일	15초마다 touch. 컨테이너 healthcheck용. DB와 별개

6.4 ActiveJobExistsError

같은 프로젝트에 queued/running이 있으면 enqueue가 거부된다. 애플리케이션 SELECT 1 ... status IN ('queued','running')과 UNIQUE 부분 인덱스가 이중으로 막는다. REST는 RPC 예외 active_job_exists.

6.5 진행 메시지 (UI가 아는 코드)

`ctx.report(progress, message)`의 `message`가 `jobs.message`에 들어간다. 프론트 `jobMessageLabel`이 `src/lib/job-labels.ts`로 번역한다. 파이프라인이 실제로 넣는 코드:

잡	message (코드 순서)	대략 progress
transcribe	measuring_duration, extracting_audio, asr, segment_timing, diarization?, correct_asr?, translate, prepare_segments, refine_timing, done	0.03 ... 1.0
dub	queued, extracting_audio, stem_split, dub_voice_tts, tts (세그먼트 루프), voice_removal, mix_bgm, burn_subtitles?, mux, done	0.02 ... 1.0
lipsync	lipsync_submit, lipsync_upload, done	0.05 / 0.9 / 1.0

라벨 테이블과 불일치

`job-labels`에는 `align_translations`, `voice_removed_preview`, `extract_vocals`, `inpaint_ambient`, `voice_clone_tts`, `voice_changer`, `prepare_asr`가 있다. 현재 `pipeline.py`는 이 중 다수를 report하지 않는다. 반대로 `voice_removal`은 워커가 쓰지만 라벨 맵에 없어 UI가 원문을 보여 준다. `cancelled` 상태 라벨도 맵에 없다.

6.6 오류 코드

`api/app/worker/errors.py`. `jobs.error`는 `{code}: {message}`. UI `formatPipelineError`는 콜론 앞 코드를 본다.

`source_missing`, `source_download_failed`, `source_too_large`, `source_too_long`, `source_unsupported_container`, `source_no_audio`, `probe_failed`, `ffmpeg_failed`, `demucs_failed`, `mix_failed`, `asr_failed`, `translation_failed`, `no_segments`, `voice_clone_failed`, `voice_missing`, `tts_failed`, `lipsync_failed`, `feature_unavailable`, `upload_failed`, `config_missing`, `internal_error`. 재퍼는 `worker timeout` (코드 형식 아님).

7. 크레딧 원장 데이터 모델

요금 화면·팩 가격은 FRS-001. 여기서는 행과 불변식만 적는다. 쓰기는 서버만.

7.1 잔액

COALESCE(sum(delta_minutes), 0). 캐시 컬럼 없음. 차감 전 profiles 행 FOR UPDATE로 직렬화한다.

7.2 reason별 행

reason	부호	생성 지점	idempotency_key
signup_grant	+30	handle_new_user. config signup_credit_minutes=30과 맞춤. 초기 마이그 레이션은 +10이었음	없음 (on conflict do nothing)
dub_job	음수	enqueue와 같은 트랜잭션. kind!=lipsync	job:{id}:debit
lipsync_job	음수	동일, kind=lipsync	동일 debit 키
refund	양수 (차감 액)	finish_job_with_refund (failed/cancelled). RC REFUND는 구매분 회수(음수)	job:{id}:refund 또는 revenuecat-revoke:...
purchase	양수	Stripe/RevenueCat 이벤트 처리	stripe-credit:... / revenuecat-credit:{tx}
admin_adjust	±	add_credit_entry + admin_note, adjusted_by	없음

7.3 불변식

- delta_minutes ≠ 0.
- 행 UPDATE/DELETE 금지. 프로젝트 삭제 시 project_id/job_id NULL만.
- 잡당 dub_job·lipsync_job·refund 각 1행 (부분 유니크).
- 잔액 부족이면 InsufficientCreditsError / insufficient_credits. 잡 행을 만들지 않음.
- charge_minutes=0이면 원장 행 없음 (transcribe).
- 이미 끝난 잡에 finish를 다시 호출하면 RPC가 no-op.
- ledger FK 때문에 하드 삭제가 막히면 프로젝트를 소프트 삭제한다.

8. R2 객체 키와 DB 참조

버킷 기본 이름 `dubby`. API는 바이트를 프록시하지 않는다. 키는 서버가 `users/{user_id}/` 아래로만 만든다.

```
users/{user_id}/
  projects/{project_id}/
    source/{filename}          <- projects.source_key
    outputs/dub_{target_lang}.mp4 <- projects.output_key
    outputs/lipsync_{lang}.mp4  <- projects.lipsync_output_key
    meta/speech_ranges.json     <- 컬럼 없음. source_key에서 유도
    meta/voice_removed.mp4      <- 선택 미리보기. 현재 transcribe는 생성 생략
    meta/dub_voice_ids.json     <- 사이드카 (voice_ids, voice_mode, pipeline_version)
    meta/dub_voice_manifest.json <- source_end_ms, speak_speed, emotion, audio_key
    meta/dub_seg_{idx}.wav      <- 편집기 미리보기 클립
    voices/inbox/{uuid}/{filename} <- 클론 업로드. 완료 후 삭제. DB 컬럼 없음
```

유도 함수	규칙
<code>source_key()</code>	<code>users/{uid}/projects/{pid}/source/{sanitize}</code>
<code>output_key_for_source()</code>	<code>source_key</code> 에서 <code>/source/</code> 앞 prefix + <code>/outputs/{file}</code>
<code>meta_key_for_source()</code>	동일 prefix + <code>/meta/{file}</code>
<code>project_meta_key()</code>	uid/pid로 meta 직접 구성
<code>voice_clone_inbox_key()</code>	inbox에 새 uuid 디렉터리

프리사인 만료: 업로드 3600초, 다운로드 60-3600(기본 3600). 파트 크기 5-16MiB. 워커는 `download_file` / `upload_file` / `head_object` / `presign_get`을 쓴다.

9. 파이프라인 단계

오케스트레이션은 `api/app/worker/pipeline.py`. 도구 호출은 `engine.py`
(`PIPELINE_MODE=real|mock|auto`). production은 `real`만 허용. 프롬프트 본문은 후속 문서.

9.1 transcribe

프로젝트: 기존 `status` → `processing` → `ready_for_edit` (실패 시 `failed`).

```
R2 source_key
| head + size <= max_source_bytes (500MB)
| download → scratch/source.bin
v
probe + validate_source          message=measuring_duration
| duration <= 1800s, 허용 컨테이너, 오디오 트랙
| projects.duration_seconds 기록
v
extract_asr_audio                message=extracting_audio
| scratch/asr_audio.mp3
v
engine.transcribe (Gemini/Whisper) message=asr
| drafts, words, speech_ranges, full_transcript
| meta/speech_ranges.json 업로드
v
chunks = drafts 그대로          message=segment_timing
| (3.0: breath regroup / gap refill 없음)
| dedupe_boundary_overlaps
v
_fill_uncovered_asr_tail         (조건 충족 시 2차 청취)
| asr_tail.mp3, step=asr_tail
v
diarization?                    message=diarization
correct_asr?                    message=correct_asr (skip_proofread면 생략)
v
translate_batch                  message=translate
| document_context = full_transcript
| 결과 dict[idx] → target_text (assign-by-idx)
v
replace_segments                message=prepare_segments, refine_timing
v
status=ready_for_edit           message=done
```

voice-removed 미리보기(두 번째 Demucs+mux)는 코드 주석상 생략한다. Before는 원본을 쓴다.

9.1.1 assign-by-idx (3.0.12)

`gemini_client.translate_batch`는 각 항목에 `idx`를 넣고, 모델이 `translations[{{idx,text}}]`를 반환해야 한다. `parse_translation_content(openai_client)`는 기대 `idx`가 모두 있어야 하며, 없으면 `TRANSLATION_FAILED(retryable)`. 위치 배열로 빈 칸을 메우지 않는다.

1차 결과를 `idxs_needing_retranslate(translation_align.py)`가 검사한다. 대상 스크립트 없음, 긴 원문의 짧은 스텝, 짧은 원문의 장문, 슬롯 대비 과장문이면 그 `idx`만 재번역한다. 서버 워커 경로는 이 재시도 후 `postprocess`한다.

로컬 스텝(`local_step12.py`)은 파서가 더 관대하다. 전부 어긋난 1-based/상대 인덱스일 때만 위치 재매핑한다. 부분 매칭 구멍에 이웃 문장을 넣지 않는다. 주석: 그렇게 하면 `source/target`

쌍이 섞인다.

9.1.2 꼬리 재청취

_fill_uncovered_asr_tail: 첫 STT가 클립 끝 전에 멈추면 후반 타임스탬프가 비고, 번역이 있어도 더빙 때 원음이 남는다.

- duration_ms < 20000 또는 covered_end ≥ 85%이면 skip.
- tail_start = max(0, covered_end - 800). 남은 길이가 4초 미만이면 skip.
- trim → transcribe(asr_tail) → 오프셋 보정 → 기존 청크와 dedupe.
- quality_warnings: asr_tail_uncovered:{end}/{dur}, asr_tail_failed, asr_tail_empty, asr_tail_appended:{n}.

9.2 dub

프로젝트: ready_for_edit → dubbing → completed. 믹스 베드는 원본에서 소스 언어 발화 구간을 깎고, 그 자리에 TTS를 올린다. 다른 언어 구간은 passthrough.

```
list_segments_for_worker + R2 manifest 병합
| speak_speed, emotion_user_set, source_end_ms 복원
| target_text 있는 행만. source_lang passthrough 제외 → speakable
v
download + probe                                message=queued 후 extracting_audio
extract_audio → audio.wav
v
split_stems (Demucs htdemucs)    message=stem_split
| vocals / no_vocals
| loudness는 vocals + speech_ranges (세그먼트 전체 구간 사용 안 함)
v
resolve voices                        message=dub_voice_tts
| Voice Box ID 또는 auto_clone IVC (vocals 샘플)
v
병렬 TTS (tts_concurrency 기본 4)    message=tts
| ElevenLabs v3, speak_speed 클램프 0.7-1.2
| 선택 voice_changer_after_tts
| 안 맞으면 compress → slot extend → speak_speed_fit
| source_end_ms는 연장하지 않음
v
fit_clip (atempo / rubberband) + 음량 매칭
persist_dub_voice_assets
v
remove_recognized_speech            message=voice_removal
| no_vocals_in_mask = 0.8 (config.mix_no_vocals_in_mask)
v
mix(selective_bed, placed_clips) message=mix_bgm
build_ass + mux                    message=burn_subtitles? / mux
upload outputs/dub_{lang}.mp4      output_key
status=completed                    message=done
```

타이밍 재작성 문장은 자막 테이블의 target_text를 바꾸지 않는다(편집기 번역 유지). end_ms와 speak_speed는 persist한다. 실패 시 클론 보이스는 cleanup_voice로 정리(Voice Box ID는 보호).

9.3 lipsync

코딩되어 있다. 기본 lipsync_provider=disabled. 더빙 output_key 필수. 원본 비디오 URL + 더빙 오디오 URL을 provider.render에 넘기고, 결과를 outputs/lipsync_{lang}.mp4에 올린 뒤 lipsync_output_key만 갱신한다. 프로젝트 status는 completed를 유지한다. mock은 더빙 파일을 file://로 넣는다. timeout/poll: sync_timeout_seconds 900, sync_poll_interval_seconds 5.

9.4 엔진 한도 (config)

키	기본
max_source_duration_seconds	1800
max_source_bytes	500 MiB
allowed_source_containers	mp4,mov,m4a,3gp,3g2,mj2,matroska,webm
demucs_model / device / jobs	htdemucs / cpu / 2
mix_no_vocals_in_mask	0.8
translation_batch_size	12
translation_timing_tolerance	0.22
tts_concurrency	4
tts_max_speedup / tts_min_tempo / tts_atempo_max	1.5 / 0.85 / 1.5

10. 스크래치·정리·재시도·취소

10.1 스크래치

`_make_scratch`: 부모는 `scratch_dir` 또는 시스템 `temp`. 디렉터리명 `dubby-job-{job_id}-`. 파일 예: `source.bin`, `asr_audio.mp3`, `asr_tail.mp3`, `audio.wav`, `stems/`, `clips/seg_{idx}.*`, `speech_removed_bed.wav`, `mixed.wav`, `subtitles.ass`, `output.mp4`, `lipsync.mp4`.

`_cleanup_scratch`: `rmtree`를 최대 2회(0.5초 간격). Windows 잠금 대비. 그래도 남으면 `warning`. 모든 핸들러 `finally`에서 호출.

10.2 재시도

`_with_retries`: `pipeline_step_retries` + 1회(기본 3). `PipelineError.retryable`만. 백오프 `pipeline_retry_backoff_seconds * attempt(2s, 4s, ...)`. 재시도 사이 `check_cancelled`. 네트워크/R2/일부 AI 단계가 `retryable`. 검증 오류(너무 김, 컨테이너, `no_segments` 등)는 한 번만.

10.3 취소 체크포인트

`ctx.report`와 `ctx.heartbeat`가 `get_job_status`를 본다. `running`이 아니면 `JobCancelled`. 러너는 이 예외에서 `finish_job`을 호출하지 않는다. `SIGTERM/SIGINT`는 신규 `claim`을 멈추고 진행 중 잡은 끝낸다.

11. 3.0.12 데이터 합의 — 재추출이 필요한 이유

VERSION.md: 3.0.12는 잘못 할당된 캡션을 재번역하고, 덮이지 않은 오디오 꼬리를 다시 들어
긴 클립이 원어로 남는 것을 줄인다. 이는 UX 문구가 아니라 **이미 저장된 행** 문제다.

11.1 잘못된 세그먼트 할당

세그먼트 진실은 `segments.target_text`의 idx 대응이다. 구버전은 모델이 문서 앞부분이나 다
른 장면을 한 idx에 넣거나, 짧은 줄에 장문을 붙일 수 있었다. 그 행은 편집기에 그대로 남고,
더빙은 그 텍스트를 읽는다.

3.0.12 워커는 *새 transcribe* 잡에서만 재번역 휴리스틱을 돌린다. 기존 행을 훑는 마이그레이
션 잡이나 UPDATE 패치는 없다. 고치는 방법은 transcribe를 다시 돌려
`replace_segments(DELETE+INSERT)`하는 것이다. 편집기에서 한 줄씩 고칠 수는 있으나, 후반부
통째 어긋남은 수작업으로 복구하기 어렵다.

11.2 후반 타임스탬프 부재

첫 Gemini 패스가 `covered_end`에서 멈추면 그 이후 청크가 DB에 없다. 번역 문자열이 문서 컨
텍스트에 있어도 세그먼트 행이 없으면 더빙 `speakable` 구간이 없고, 후반은 원음/뮤트 베드가
된다. `speech_ranges.json`도 같은 패스에서 잘리면 음량·voice removal이 후반을 놓친다.

꼬리 재청취는 실행 중 transcribe에만 적용된다. 이미 `ready_for_edit`인 프로젝트의
`segments.R2` 메타는 자동 수리되지 않는다. 재추출이 필요한 이유:

- 세그먼트는 전량 교체 모델이다. 부분 append API가 없다.
- `source_end_ms`는 DB에 없어 매니페스트가 비면 ASR 끝을 복원할 수 없다.
- `dub`은 있는 행의 `start_ms/end_ms`만 본다. 없는 분을 만들어 주지 않는다.

12. 마이그레이션 / 부트스트랩

infra/DEPLOY.md:

```
supabase link --project-ref <PROJECT_REF>
supabase db push
```

또는 SQL editor에 마이그레이션을 시간순으로 붙여 넣는다. 초기 파일만 넣으면 Stripe·언어·Voice Box·th/my·감정 톤 CHECK가 빠진다. API는 빠진 projects 컬럼을 SELECT에서 생략하지 만, th/my 생성은 CHECK 위반으로 400이 난다(라우터 안내: 해당 마이그레이션 적용).

Lightsail은 스키마를 적용하지 않는다. Docker API/워커는 DATABASE_URL(postgres) 또는 service role(REST)로 이미 있는 DB에 붙는다. production은 SUPABASE_URL과 백엔드에 맞는 URL/키가 없으면 기동 실패.

첫 관리자: 20260719165000_bootstrap_first_admin.sql이 이메일 1건을 요구하고 raw_app_meta_data.role=admin을 넣는다. 건수가 1이 아니면 예외.

신규 사용자: Auth INSERT → profiles + credit_ledger signup_grant 30. 트리거와 config가 어긋나면(과거 +10) 실제 지급은 트리거 값이 이긴다.

13. 명시적 비범위

- Caddy, CORS, Cloudflare Tunnel, GitHub Pages 토폴로지 — SAD-001.
- Stripe Checkout·빌링 화면, 크레딧 팩 UX, 언어 피커 — FRS-001.
- Gemini/OpenAI/ElevenLabs 프롬프트 전문, 스키마 JSON, 모델 선택 이유 — 후속 프롬프트 명세서.
- 존재하지 않는 테이블(예: generic webhooks, segment_status, credit_balances)은 설계에 넣지 않았다.

14. 개정 이력

버전	날짜	내용
1.0	2026-09-05	코드 3.0.12 / 파이프라인 3.0 기준 최초 DPD